

Wykorzystanie oprogramowania do zarządzania konfiguracją

Git | Ansible | Puppet | Chef | Terraform | SaltStack

Igor Zaworski 280631
Mariusz Rychlicki 280451

Wspomaganie Zarządzania Projektami IT

8 czerwca 2026

Plan prezentacji

- 1 Wprowadzenie – czym jest zarządzanie konfiguracją?
- 2 Typologia i kategorie narzędzi CM
- 3 Przegląd narzędzi: Git, Ansible, Puppet, Chef, Terraform, SaltStack
- 4 Porównanie funkcjonalne i cenowe
- 5 Zalety i wady każdego rozwiązania
- 6 Podsumowanie korzyści
- 7 Ćwiczenia praktyczne: Git i Ansible

Co to jest zarządzanie konfiguracją?

Definicja

Zarządzanie konfiguracją (CM) to proces śledzenia i kontrolowania zmian w oprogramowaniu, infrastrukturze i środowiskach systemowych.

- Wywodzi się z praktyk ITIL i wojskowych standardów zarządzania
- W erze DevOps stało się fundamentem automatyzacji IT
- Umożliwia **Infrastructure as Code** (IaC)
- Kluczowe pojęcia: *idempotentność*, *desired state*, *drift detection*

Problem bez CM

„Działa na moim komputerze” – brak reproducibility, ręczne konfiguracje, brak audytu, “snowflake servers”

Cele strategiczne

- Spójność środowisk (dev/test/prod)
- Redukcja błędów ludzkich
- Audytowalność zmian
- Szybki rollback
- Dokumentacja infrastruktury w kodzie

Funkcje operacyjne

- Automatyzacja wdrożeń
- Egzekwowanie stanu konfiguracji
- Wykrywanie dryftu konfiguracji
- Zarządzanie sekretami
- Raportowanie zgodności

Korzyści biznesowe

Szybsze wdrożenia $\Rightarrow$ niższe koszty operacyjne $\Rightarrow$ wyższe SLA

Model Push

- Kontroler wysyła konfigurację
- Natychmiastowe działanie
- Przykłady: Ansible, Terraform

Agentless

- Komunikacja przez SSH/WinRM
- Brak dodatkowego oprogramowania
- Łatwiejsze wdrożenie

Model Pull

- Agent sam pobiera konfigurację
- Ciągłe egzekwowanie stanu
- Przykłady: Puppet, Chef

Agent-based

- Demon działa na węźle
- Lepsza skalowalność
- Stały monitoring stanu

- **Rok:** 2005, Linus Torvalds (dla jądra Linux)
- **Typ:** Rozproszony system kontroli wersji (DVCS)
- **Licencja:** GPL v2 (darmowy)
- **Hostingi:** GitHub, GitLab, Bitbucket

Kluczowe funkcje:

- Śledzenie historii zmian w plikach (diff, blame)
- Rozgałęzianie (*branching*) i łączenie (*merging*)
- Praca offline – pełne repozytorium lokalnie
- Podstawa dla CI/CD i GitOps

Zastosowanie

- Kod źródłowy
- Pliki konfiguracyjne
- Dokumentacja
- Playbooki Ansible
- Moduły Terraform

- **Rok:** 2012; Red Hat przejął w 2015
- **Typ:** Agentless CM, model push
- **Licencja:** Apache 2.0 (Community)
- **Język:** YAML (playbooki)

Kluczowe funkcje:

- Komunikacja przez SSH – brak agentów
- Idempotentne moduły (ponad 3 000)
- Role i kolekcje (Ansible Galaxy)
- Ansible Tower/AWX – interfejs webowy
- Obsługa sieci, chmury i kontenerów

Zastosowanie

- Konfiguracja serwerów
- Deployments
- Orkiestracja
- Patch management
- Provisioning VM

- **Rok:** 2005, Puppet Labs (obecnie Perforce)
- **Typ:** Agent-based CM, model pull
- **Licencja:** Apache 2.0 (CE) / Puppet Enterprise
- **Język:** Puppet DSL (deklaratywny)

Kluczowe funkcje:

- Deklaratywny – opisujesz *stan*, nie kroki
- Puppet Master + Agent (katalog zasobów)
- PuppetDB – historia zmian i faktów
- Silne raportowanie i audyt
- Hiera – hierarchia danych konfiguracyjnych

Zastosowanie

- Duże enterprise
- Stała zgodność
- Compliance
- Setki węzłów

- **Rok:** 2009, Opscode; Progress Software od 2020
- **Typ:** Agent-based CM, model pull
- **Licencja:** Apache 2.0 (CE) / Progress Chef
- **Język:** Ruby DSL (cookbooks, recipes)

Kluczowe funkcje:

- Chef Server, Workstation, Client (node)
- Test Kitchen – automatyczne testy konfiguracji
- InSpec – testy zgodności (compliance as code)
- Supermarket – społecznościowe cookbooks

Zastosowanie

- Złożone logiki
- DevSecOps
- Compliance testing
- Finanse i banki

- **Rok:** 2014, HashiCorp
- **Typ:** IaC Provisioning, model push
- **Licencja:** MPL 2.0 (CE) / HCP
Terraform
- **Język:** HCL (HashiCorp Configuration Language)

Kluczowe funkcje:

- Tworzenie infrastruktury
- Ponad 3 000 providerów (AWS itd.)
- plan → apply – podgląd zmian przed wdrożeniem
- Zarządzanie stanem
(`terraform.tfstate`)
- Fork OpenTofu (BSL od 2023)

Zastosowanie

- Chmura
(AWS/Azure/GCP)
- Multi-cloud
- Kubernetes
- Sieci i DNS

- **Rok:** 2011; VMware przejął w 2020
- **Typ:** CM + Event-driven automation
- **Licencja:** Apache 2.0 (CE) / VMware Aria
- **Język:** YAML + Jinja2

Kluczowe funkcje:

- Szybka komunikacja przez ZeroMQ
- Salt Master + Salt Minion (lub SSH agentless)
- Reaktory i zdarzenia – automatyczne odpowiedzi
- Obsługa tysięcy węzłów równolegle
- Grains i Pillars – dane o węzłach

Zastosowanie

- Duże infrastruktury
- Event-driven ops
- Cloud management
- VMware/vSphere

Porównanie funkcjonalne

Narzędzie	Kategoria	Model	Agentless	Język/DSL	Krzywa uczenia
Git	SCM	distributed	✓	–	Średnia
Ansible	CM	push	✓	YAML	Niska
Puppet	CM	pull	✗	Puppet DSL	Wysoka
Chef	CM	pull	✗	Ruby DSL	Bardzo wysoka
Terraform	IaC	push	✓	HCL	Średnia
SaltStack	CM	push/pull	opcja	YAML/Jinja2	Wysoka

Narzędzie	Platformy	Integracja z chmurą
Git	Wszystkie systemy	GitHub Actions, GitLab CI, Azure DevOps
Ansible	Linux, Windows, urządzenia sieciowe	AWS, Azure, GCP, OpenStack
Puppet	Linux, Windows, macOS	AWS, Azure, GCP
Chef	Linux, Windows, macOS	AWS, Azure, GCP
Terraform	API/chmura (nie OS)	AWS, Azure, GCP, Oracle, IBM i in.
SaltStack	Linux, Windows	AWS, Azure, GCP, VMware

Porównanie rynkowe i cenowe

Narzędzie	Licencja (CE)	Wersja Enterprise	Cena Enterprise
Git	GPL v2	-	-
Ansible	Apache 2.0	Red Hat AAP	~\$14 000/rok (100 węzłów)
Puppet	Apache 2.0	Puppet Enterprise	~\$112/węzeł/rok
Chef	Apache 2.0	Progress Chef	~\$130/węzeł/rok
Terraform	MPL 2.0	HCP Terraform Plus	\$20/user/mies.
SaltStack	Apache 2.0	VMware Aria Automation	kontraktowo

Ważna uwaga

Wszystkie narzędzia mają darmowe wersje open source. Koszty enterprise rosną wraz z liczbą węzłów/użytkowników. Przy 100+ węzłach koszt może przekroczyć \$10 000/rok.

Zalety i wady – Git, Ansible, Puppet

Narzędzie	Zalety	Wady
Git	Darmowy; rozproszony (offline); szybki branching; ogromna społeczność; standard branżowy; wersjonowanie wszystkiego	Konflikty merge; krzywa uczenia (rebase, cherry-pick); trudne zarządzanie dużymi plikami binarnymi (wymaga Git LFS)
Ansible	Agentless (tylko SSH); YAML łatwy do nauki; ponad 3000 modułów; Ansible Galaxy; szybki start; idempotentność	Wolniejszy przy tysiącach węzłów; push – brak ciągłego monitoringu stanu; brak natywnego GUI (wymaga Tower/AWX)
Puppet	Dojrzałe rozwiązanie enterprise; silny model pożądanego stanu; dobre raportowanie; PuppetDB; Hiera	Wymaga agenta; trudny własny DSL; drogie licencje Enterprise; mniejsza społeczność niż Ansible

Zalety i wady – Chef, Terraform, SaltStack

Narzędzie	Zalety	Wady
Chef	Bardzo elastyczny (pełny Ruby); silne testowanie (Test Kitchen); InSpec dla compliance; dojrzałe rozwiązanie	Duży próg wejścia (Ruby); złożona architektura; mniejsza społeczność; kosztowne licencje enterprise
Terraform	Provider-agnostic; deklaracyjny; plan przed apply; ogromny ekosystem providerów; standard dla chmury	Zarządzanie stanem (remote backend); nie konfiguruje OS; zmiana licencji BSL (2023) – powstał fork OpenTofu
SaltStack	Bardzo szybki (ZeroMQ); event-driven; elastyczny (push i pull); świetny dla dużych infrastruktur; reaktory	Mniejsza popularność niż Ansible; złożona konfiguracja; słabsza dokumentacja; niepewna przyszłość po przejęciu przez VMware

Podsumowanie korzyści

Dla programisty / admina

- Automatyzacja powtarzalnych zadań
- Wersjonowanie infrastruktury jak kodu
- Szybki rollback do poprzedniego stanu
- Reproducibility – identyczne środowiska
- Mniej pomyłek, mniej pracy ręcznej

Dla organizacji

- Niższe koszty operacyjne (OPEX)
- Szybsze wdrożenia – wyższy TTM
- Zgodność z regulacjami (compliance)
- Wyższa dostępność (SLA/SLO)
- Dokumentacja infrastruktury w kodzie

Rekomendacja na 2025

Wybor narzędzia zależy od skali, chmury i kompetencji zespołu.

Najczęstszy zestaw: **Git + Terraform + Ansible**

Podstawowe komendy

- `git init` / `git clone <url>`
- `git add .` / `git commit -m`
- `git push` / `git pull`
- `git branch <name>` / `git checkout`
- `git merge` / `git rebase`
- `git log -oneline -graph`
- `git stash` / `git tag v1.0`

Workflow GitFlow

- `main` – produkcja (stabilna)
- `develop` – integracja
- `feature/X` – nowe funkcje
- `release/X` – przygotowanie
- `hotfix/X` – pilne poprawki

Dobre praktyki

- Conventional Commits
- `.gitignore` dla sekretów
- Pull Requests + code review

Struktura projektu

- inventory/ – lista hostów
- playbooks/ – główne playbooksi
- roles/ – wielokrotnego użytku
- group_vars/ – zmienne grup
- ansible.cfg – konfiguracja

Ad-hoc commands

```
ansible all -m ping
```

```
ansible web -m apt -a name=nginx"
```

```
ansible-playbook site.yml
```

Przykładowy playbook YAML

```
--  
- hosts: webservers  
  become: yes  
  vars:  
    pkg: nginx  
  tasks:  
    - name: Install pkg  
      apt:  
        name: " pkg "  
        state: present  
    - name: Start pkg  
      service:  
        name: " pkg "  
        state: started  
        enabled: yes
```

Dziękuję za uwagę

Pytania?

Temat: Zarządzanie konfiguracją w IT
Narzędzia: Git, Ansible, Puppet, Chef, Terraform, SaltStack
Autor: Igor Zaworski 280631 Mariusz Rychlicki 280451